Threat Intelligence API Specification & Integration Guide

Enterprise Developer Manual for Customer Ingestion, SIEM/SOC Feeds, and Automated Pipelines

Executive Summary: This document provides the complete technical specification for consuming the **Telegram Darknet Cyber Threat Intelligence (CTI) API (v1)**. The system continuously scrapes over 28 monitored underground Telegram threat actor channels, executing real-time NLP classification, severity scoring, and indicator extraction (malicious URLs, crypto wallets, CVE exploits, and forensic evidence). Through this REST API, external security operation centers (SOCs), SIEM platforms (Splunk, Microsoft Sentinel), and customer automated scripts can ingest high-fidelity threat telemetry programmatically.

1. System Architecture & High-Level Ingestion Flow

The architecture is built upon a defense-in-depth model that ensures high availability, strict client isolation, and total protection of the core database infrastructure:

Component	Role	Security & Protocol Details
Scraper Core	Autonomous Crawler	Monitors 28+ channels 24/7; extracts chat text, media, metadata.
Forensic LLM & Regex	Analysis Pipeline	Extracts IOCs (CVEs, IPs, BTC/ETH wallets, hashes, phishing links).
Database Tier	Primary Storage	Encrypted MongoDB + SQLite fallback. Direct customer access is blocked.
API Gateway (FastAPI)	Customer Endpoint	Enforces API key auth, per-key rate limits, request telemetry.
Cloudflare Tunnel	Secure Egress Gateway	Zero open router ports; DDoS shield; TLS 1.3 encrypted transit.

2. Authentication, Rate Limiting & Security Governance

Every request to the Customer API must be authenticated using a customer-specific API key possessing the dnet_live_ prefix. The key can be passed via three supported mechanisms:

HTTP Header (Recommended)	X-API-Key: dnet_live_<token>	Production SIEM, Python, Node.js, Go scripts
Bearer Authorization	Authorization: Bearer dnet_live_<token>	Standard REST/OAuth2 compatible clients

URL Query Parameter	?api_key=dnet_live_<token>	Ad-hoc browser inspection and curl testing

Key Security Guarantees for Infrastructure Protection:

• **Strict Read-Only Access:** All customer endpoints expose solely HTTP GET methods. External keys have zero write, update, or deletion privileges on the database.

• **Administrative Guard (AdminGuard):** Management endpoints (key creation, quota editing, telemetry reset) are strictly restricted to internal localhost/LAN access and are automatically blocked for tunnel callers.

• **Sliding-Window Rate Limiting:** Each key is capped at 120 requests/minute. Excessive traffic triggers an HTTP 429 Too Many Requests response with a dynamic Retry-After countdown header.

• **Cryptographic Hashing:** API keys are stored exclusively as one-way SHA-256 hashes in MongoDB. Cleartext keys are never retrievable from the database.

3. Customer API Endpoint Reference (v1)

Base Live Production URL: https://set-represents-suggested-leader.trycloudflare.com
Local Development Base URL: http://localhost:8000

Method	Endpoint Path	Parameters & Filters	Description / Output
GET	/api/v1/threats	min_level (ALL, LOW, MED, HIGH, CRITICAL) limit (1-200, default 50) page, keyword, sector	Fetches real-time darknet threat alerts and scraped messages sorted chronologically with full severity tags.
GET	/api/v1/threats/{id}	id: Message ID string	Retrieves full forensic details, IOCs, entities, and screenshot evidence for a specific message.
GET	/api/v1/iocs	type (all, wallet, url, cve, ip) limit (1-500, default 100)	High-speed stream of extracted Indicators of Compromise optimized for firewall and SIEM blocklist ingestion.
GET	/api/v1/stats	None	Provides high-level CTI metadata, channel counts, active monitoring metrics, and severity distributions.

Sample API Response Payload (GET /api/v1/threats?limit=1):

4. How It Will Be Integrated (Practical Production Patterns)

Customers and internal stakeholders can integrate the CTI feed into their infrastructure through any of the following proven architectural models:

Integration Pattern 1: Automated Continuous Ingestion Pipeline (Python / Worker)

Ideal for internal security data lakes, threat analysis databases, or custom analyst alerting systems. The script queries the API every 60 seconds, gracefully handles rate limits, and ingests newly posted intelligence.

```
import time, requests

API_URL = "https://set-represents-suggested-leader.trycloudflare.com/api/v1/threats"
HEADERS = {"X-API-Key": "dnet_live_YOUR_CUSTOMER_KEY"}

def fetch_latest_threats():
    params = {'limit': 50, 'min_level': 'HIGH'}
    try:
        resp = requests.get(API_URL, headers=HEADERS, params=params, timeout=15)
        if resp.status_code == 200:
            threats = resp.json().get('threats', [])
            for threat in threats:
                # Ingest into customer database or Slack/PagerDuty notification
                print(f"[{threat['threat_level']}] {threat['channel']}: {threat['id']}")
        elif resp.status_code == 429:
            wait_sec = int(resp.headers.get('Retry-After', 10))
            print(f'Rate limit reached. Backing off for {wait_sec}s.')
            time.sleep(wait_sec)
    except requests.RequestException as err:
        print(f'Network error: {err}')
```

Integration Pattern 2: Enterprise SIEM / SOAR Ingestion (Splunk, Sentinel, QRadar)

SOC teams configure their SIEM's Threat Intelligence Ingestion Add-on or HTTP Event Collector (HEC) to poll the /api/v1/iocs endpoint on an hourly or daily schedule. Extracted indicators (malicious domains, ransomware wallet addresses, CVE identifiers) are immediately pushed to firewalls (Palo Alto, Fortinet) and Web Application Firewalls (WAFs) for automated zero-touch blocking.

Integration Pattern 3: Custom Customer Portal / Microservice (Node.js / TypeScript)

```
// Node.js 18+ native fetch example
const response = await fetch('https://set-represents-suggested-leader.trycloudflare.com/api/v1/threa
ts?limit=25', {
  headers: {
    'X-API-Key': 'dnet_live_YOUR_CUSTOMER_KEY',
    'Accept': 'application/json'
  }
});
const data = await response.json();
console.log(`Retrieved ${data.threats.length} threat records.`);
```

5. Step-by-Step Verification Protocol for Leadership & QA

To satisfy project stakeholders and confirm the deployment readiness of the API, execute the following four-stage validation checklist:

Step	Validation Objective	Action & Command	Expected Result
1	Interactive Swagger Exploration	Navigate to /docs in browser, click **Authorize**, enter API Key, execute GET /api/v1/threats.	HTTP 200 OK with live Telegram threat JSON payload.
2	Command-Line Tool Validation	Run python fetch_threats.py dnet_live_YOUR_KEY from any workstation across the web.	Formatted terminal output listing the top 100 messages.
3	Security & Auth Rejection	Pass an invalid token: curl -H 'X-API-Key: bad_key' <URL>/api/v1/threats.	HTTP 401 Unauthorized with descriptive JSON error.
4	Telemetry & Metrics Verification	Check Web Dashboard at /api-keys after running steps 1 & 2.	The *Usage & Requests* counter increments from '0 calls' to '2 calls'.

Document Sign-off & Support:
- **Engineering Team:** Telegram Darknet Threat Intelligence Engineering Division
- **API Version:** 1.0 (Stable)
- **Support Contact:** Security Operations Center / Threat Intelligence Lead
- **Classification:** TLP:WHITE — Approved for Customer Distribution & Partner Integration